



Atelier : Tout sur les agents
Jérémy VANGANSBERG
sept-2025

Plan

1. Pourquoi des agents ?
2. Mécanique d'un agent (les bases opérationnelles)
3. ReAct, Function Calling, Extraction structurée (démonstration)
4. De l'agent au multi-agents
5. Écosystème & front (panorama outillage)
6. Focus sur LangGraph (démonstration)
7. Sujets avancés & recommandations



Lien github : https://github.com/jeremy-vangansberg/agents_demo

Pourquoi les agents ?

Promesses & limites des LLM

Ce qu'ils font très bien (exemples)

– Résumé • Extraction d'informations • Reformulation • Traduction • Classement rapide

Limites en usage de base (et pourquoi)

1. **Connaissance figée**
2. **Calculs approximatifs**
3. **Isolation de l'environnement**
4. **Hallucination**

Comment améliorer les performances des LLM ?

Mesurer les performances

Les [benchmarks](#) sont des tests objectifs (comme des QCM) pour mesurer la performance d'un modèle sur des tâches spécifiques.

Exemples : GSM8K (mathématiques), MMLU (compréhension), HumanEval (code)

Principales familles de techniques

Il existe plusieurs techniques pour améliorer les performances d'un modèle :

Techniques de raisonnement

- **CoT** (Chain of Thought), **ToT** (Tree of Thoughts)
- **Objectif** : Améliorer la logique et le raisonnement

Techniques d'action

- **Function calling, Tool use**
- **Objectif** : Interaction avec l'environnement externe

Techniques hybrides

- **ReAct** (Reasoning + Acting), **Plan-and-Execute**
- **Objectif** : Combiner raisonnement et action

Modes d'implémentation

Deux modes d'implémentation

Ces techniques peuvent être effectuées avec ET sans fine-tuning.

Avec fine-tuning

- Optimisation spécifique pour des cas d'usage particuliers
- Amélioration des performances sur des domaines ciblés

Sans ré-entraînement

Comment modifier le comportement d'un modèle sans le ré-entraîner ?

- **Prompt engineering & Context engineering** : Instructions structurées
- **Logique du runtime** : Orchestration dans le code

Note : réalité des modèles propriétaires

Depuis l'émergence de ChatGPT, les grands acteurs sont de plus en plus **opaques** sur le fonctionnement de leurs modèles propriétaires.

Cependant certains articles scientifiques restent ouverts comme **DeepSeek R1** qui nous indique comment il a été fine-tuné pour raisonner, notamment grâce à des tokens spéciaux `<think>` et `</think>`. On peut donc supposer que les autres modèles de raisonnement comme **OpenAI o1** utilisent des techniques similaires.

Mécanique d'un agent (les bases opérationnelles)

Intuition : qu'est-ce qu'un agent ?

"Un agent est un système qui s'appuie sur un modèle d'IA pour interagir avec son environnement afin d'atteindre un objectif défini par l'utilisateur. Il combine le raisonnement, la planification et l'exécution d'actions (souvent via des outils externes) pour accomplir des tâches."

Deux parties fondamentales :

1. 🧠 Le cerveau → le LLM ou un modèle plus spécialisé (VLM, AudioLM, etc.)

- Il **lit** une instruction en langage naturel
- Il décide quelle **action** entreprendre et avec quel **outil**
- Il peut **planifier** plusieurs étapes (ex : "je dois d'abord chercher, puis analyser")

2. 🦾 Le corps → les capacités d'action

- C'est ce que l'agent peut réellement faire (exécuter un code, chercher sur le web, envoyer un email...)
- Ces actions passent par des **outils** définis par le développeur

... On pourrait également ajouter une 3e partie importante : la mémoire.

Intuition : qu'est-ce qu'un outil ?

Définition

Un **outil** est une fonction mise à disposition du LLM pour accomplir une tâche spécifique.

L'objectif : **compléter les capacités natives** du modèle avec des fonctionnalités spécialisées.

Pourquoi utiliser des outils ?

Certaines tâches sont mieux réalisées par des fonctions dédiées que par le LLM seul.

Exemple concret : Pour des calculs arithmétiques précis, un outil calculatrice sera plus fiable que les capacités de calcul du modèle.

Exemples d'outils

- `web_search()` - Recherche sur internet
- `sql_query()` - Requêtes base de données
- `get_weather()` - Données météorologiques
- `generate_image()` - Génération d'images

Thought -> Action -> Observation

Définition

Le cycle **Pensée-Action-Observation** est un modèle théorique pour qu'un agent puisse travailler avec des outils de manière itérative.

1. **Pensée** : Le LLM de l'agent décide quelle devrait être la prochaine étape
2. **Action** : L'agent effectue une action en appelant les outils avec les arguments associés
3. **Observation** : Le modèle réfléchit sur la réponse de l'outil

Lien avec les techniques d'action

Ce cycle TAO correspond aux **techniques hybrides** () vues précédemment : il combine raisonnement ET action avec l'environnement externe. C'est ce qui distingue les **agents** des simples modèles de raisonnement.

Implémentations du cycle TAO

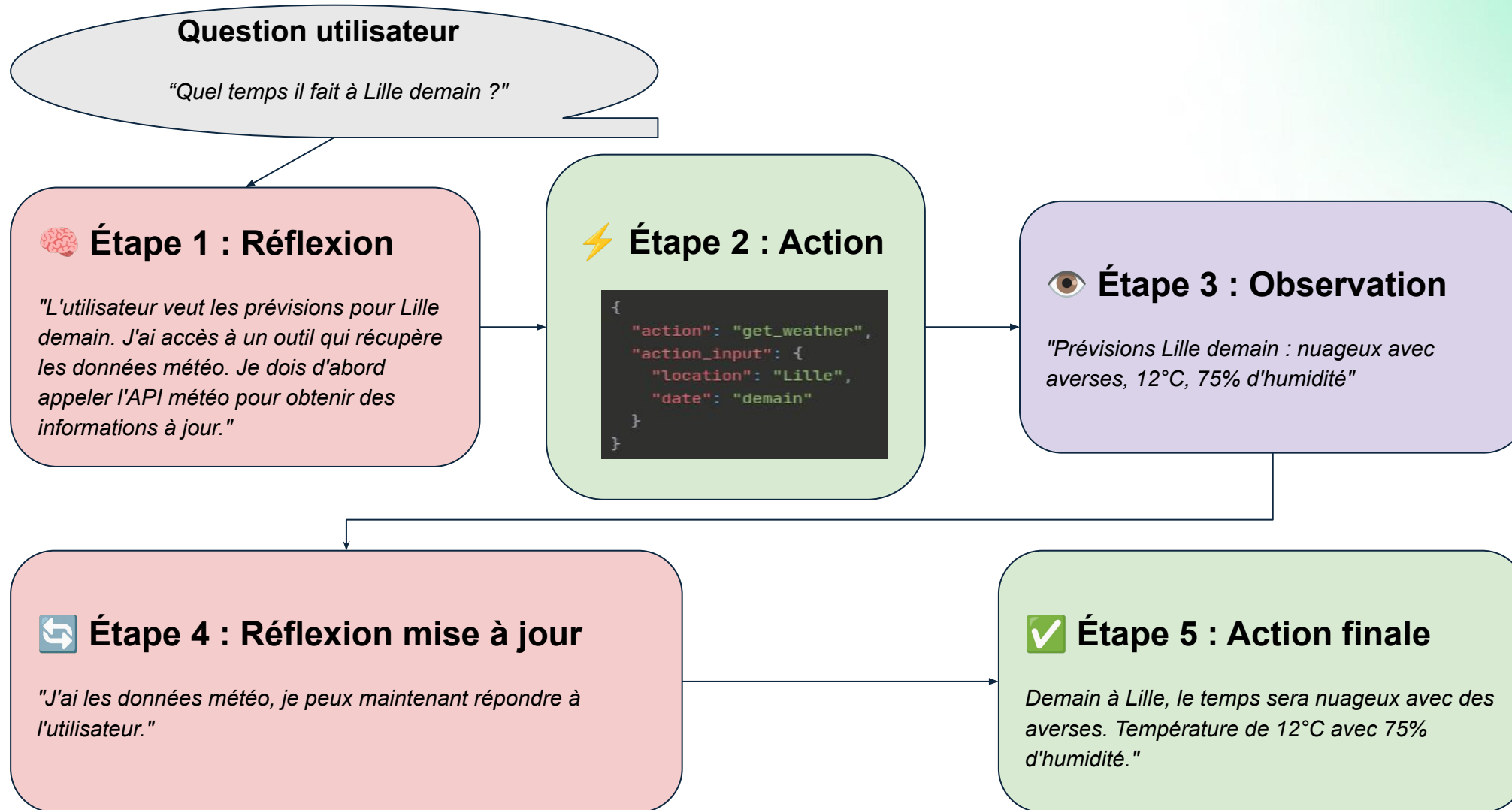
Patterns structurés issus d'articles scientifiques

- **ReAct** : Le plus connu, raisonnement + action explicites dans le prompt
- **Plan & Execute** : Planification puis exécution séquentielle

Approches minimalistes

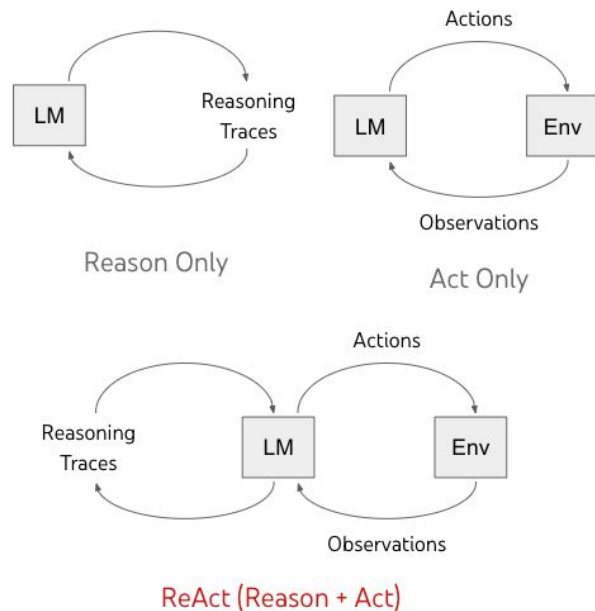
- **Function calling + boucle de contrôle** dans le runtime
- Plus simple, moins de prompting explicite, plus de logique applicative

Intuition sur le cycle Pensée-Action-Observation



ReAct, Function Calling,
Extraction structurée

ReAct : Synergizing Reasoning and Acting in Language Models



Importance historique

ReAct est un article scientifique majeur dans le développement des agents.

Titre complet

"ReAct: Synergizing Reasoning and Acting in Language Models" (2022)

Auteurs

Shunyu Yao et al. (Princeton University & Google Research)

Impact

- Premier framework structuré combinant raisonnement et action
- Base théorique de la plupart des agents modernes
- Inspiration pour LangChain, AutoGen et autres frameworks

Démonstration ReAct



Short Course • 1 hour 32 mins

AI Agents in LangGraph

AI Frameworks

Agents

Chatbots

Source : [deeplearning.ai](https://www.deeplearning.ai)

Objectif de la démo

Comprendre la bonne **répartition des responsabilités** : ce que fait le **LLM**, ce qui relève du **Prompt System**, et ce qui doit rester au **Runtime**.

1) Prompt System (conception du protocole)

- Déclare les **outils disponibles** (fonction, inputs/outputs, contraintes).
- Impose la **structure TAO** (*Thought* → *Action* → *Observation*) et le **format de réponse**.
- Utilise du **few-shot examples**

2) LLM (raisonne & propose l'action)

- Produit le **Thought** (raisonnement) et une **Action** (appel d'outil) conforme au protocole.
- S'arrête dès qu'il doit agir (
- Quand c'est fini, émet "**Final Answer:**".

3) Runtime (execution & boucle)

- **Interrompt** après l'Action, **extraie** la fonction, **exécute** l'outil.
- **Injecte l'Observation** dans le fil de conversation.

Function Calling : ce qui change

Dans l'exemple précédent, on définissait les **fonctions** directement dans le prompt système, mais depuis GPT-4 avec function calling (juin 2023), il existe une façon plus standardisée et pratique de le faire.

Quand on utilise les APIs propriétaires, on peut désormais passer la liste de nos outils (avec leurs métadonnées) directement au LLM via un paramètre dédié : le **function calling**.

Comment ça fonctionne ?

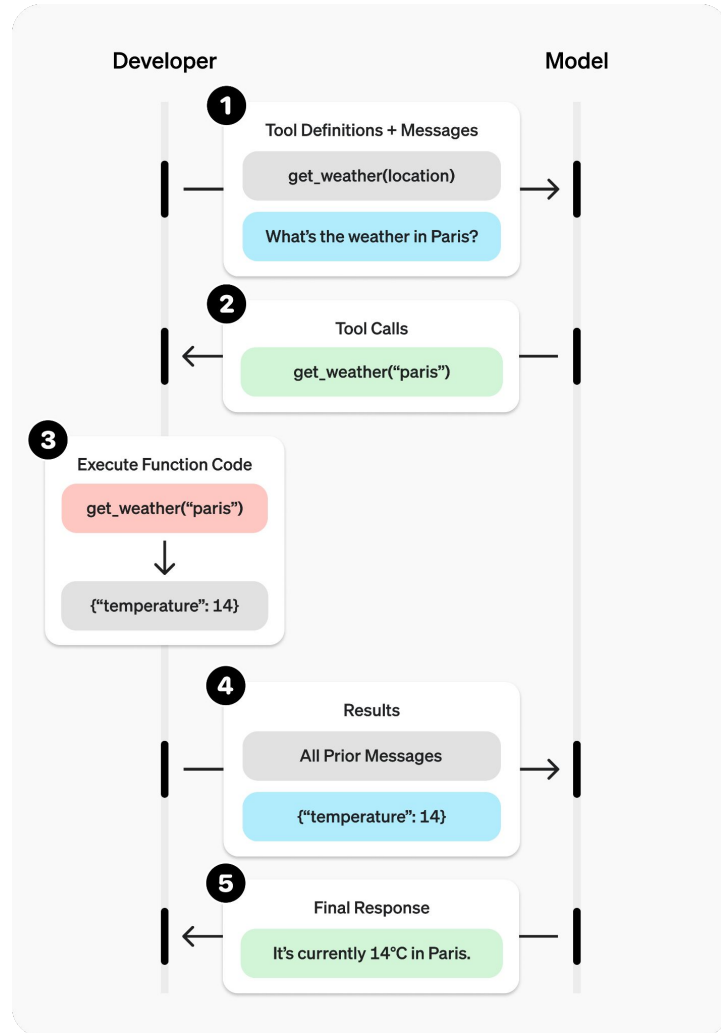
Bien qu'il n'y ait pas de communication officielle complète sur l'implémentation interne, on peut raisonnablement supposer :

1. **Entraînement spécialisé** : Les modèles ont été entraînés spécifiquement pour reconnaître et exécuter des appels de fonctions
2. **Traduction automatique** : Les providers (OpenAI, Anthropic, etc.) traduisent automatiquement les définitions de fonctions qu'on envoie dans le payload JSON en instructions optimisées intégrées au prompt système
3. **Format standardisé** : Au lieu de décrire manuellement les fonctions dans le prompt, on utilise un schema JSON standard (OpenAPI/JSON Schema) qui décrit les paramètres, types, et descriptions

Avantages de cette approche

- **Pratique** : Possibilité de définir les fonctions en Python natif (docstring, type hinting) ou avec des bibliothèques comme Pydantic (permet de faire de la validation) pour traduire automatiquement nos définitions de fonction en Python en JSON
- **Maintenance** : Plus facile de gérer et mettre à jour les outils
- **Performance** : Optimisations internes par les providers

Function Calling : détail de fonctionnement



Démonstration

Extraction d'output structuré

Le principe

Le **function calling** facilite grandement l'extraction de données structurées : au lieu de demander au LLM de respecter un format via le prompt système, on définit directement la structure attendue comme une fonction.

[Exemple](#)

Démonstration 3.structured-output

Cas d'usage pratiques

- **Variables d'état** : transmettre des données entre étapes d'un workflow
- **Boucles de clarification** : extraire progressivement un schéma cible complet
- **Parsing** : convertir emails/documents en données structurées
- **Validation de données** : s'assurer que l'extraction respecte un format attendu

Extraction d'output structuré : avancé

```
class Address(BaseModel):  
    """Adresse physique complète d'une personne"""  
    street: str = Field(description="Numéro et nom de rue")  
    city: str = Field(description="Ville")  
    postal_code: str = Field(description="Code postal")  
  
class Person(BaseModel):  
    """Informations personnelles complètes d'un individu"""  
    name: str = Field(description="Prénom et nom complets")  
    addresses: List[Address] = Field(description="Adresses connues")
```

Défis et limitations

- **Complexité croissante** : plus d'imbrications = tâche plus difficile
- **Nature stochastique** : les LLM ne réussissent pas à 100%

Facteurs d'amélioration

- **Puissance du modèle** : les modèles plus avancés suivent mieux les instructions complexes → capacité d'extraction améliorée
- **Descriptions détaillées** : Field(description="...") guide le modèle et réduit les erreurs
- **Librairies spécialisées** : mécanismes de retry automatiques et gestion d'erreurs (à voir plus tard)

Bonnes pratiques Function Calling

Source

1. Clarté et documentation

- **Noms explicites** : `get_weather(city, unit)` plutôt que `fetch(loc, type)`
- **Descriptions détaillées** : expliquer le but de chaque fonction et paramètre (format inclus), ce que retourne la fonction
- **Instructions dans le prompt système** : Quand il faut et ne faut pas utiliser les fonctions
- **Exemples et cas limites** : pour corriger les échecs récurrents (**Attention** : peut nuire aux modèles de raisonnement)

2. Bonnes pratiques de développement

- **Test du stagiaire** : un stagiaire peut-il utiliser la fonction avec juste vos instructions ? Sinon, ajoutez les réponses à ses questions dans le prompt

3. Décharger le modèle, utiliser le code

- **Ne pas faire remplir des arguments connus** : si vous avez déjà l'`order_id`, utilisez `submit_refund()` sans paramètre et passez l'ID via le code
- **Déléguer au code** : réduire la charge cognitive du modèle

4. Performance

- **Limiter le nombre** : < 20 fonctions à la fois

Du single agent au
multi-agents

Le spectre agentique

Niveau d'agence	Description	Nom de l'approche	Exemple de schéma
☆☆☆	La sortie de l'agent n'a aucun impact sur le flux du programme	Processeur simple	<code>process_llm_output(llm_response)</code>
★★☆	La sortie de l'agent détermine un flux de contrôle basique	Routeur	<code>if llm_decision(): path_a() else: path_b()</code>
★★☆	La sortie de l'agent détermine quelle fonction exécuter	Appel d'outils (Tool caller)	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
★★★	La sortie de l'agent contrôle l'itération et la poursuite du programme	Agent multi-étapes	<code>while llm_should_continue(): execute_next_step()</code>
★★★	Un workflow agentique peut en déclencher un autre	Multi-agents	<code>if llm_trigger(): execute_agent()</code>

source : huggingface agent course

Limites des Agents Uniques

Problèmes rencontrés avec un seul agent

Surcharge cognitive

- **Trop de fonctions** : > 20 outils → confusion et erreurs
- **Contexte complexe** : difficile de maintenir l'état sur de longues conversations
- **Instructions contradictoires** : prompt système surchargé

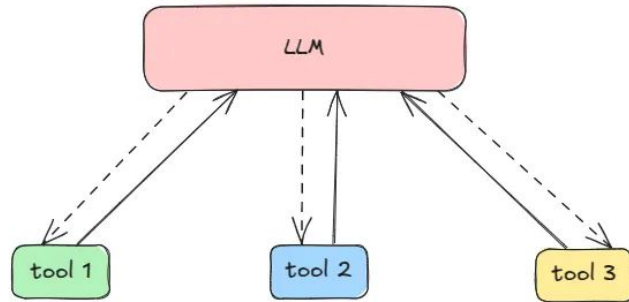
Spécialisation limitée

- **Généraliste vs Expert** : un agent qui fait tout ne fait rien parfaitement
- **Domaines différents** : difficile d'exceller en code ET en rédaction ET en analyse
- **Conflits de rôles** : être à la fois créatif et analytique

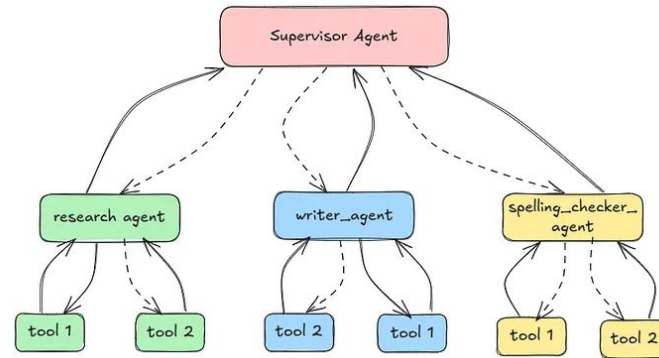
Solution : Multi-Agents

Les différents patterns

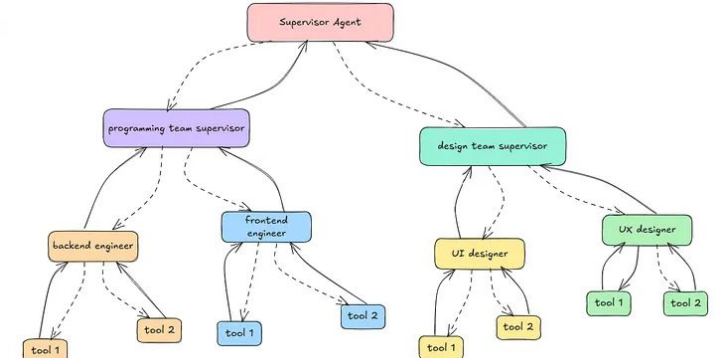
Agent unique



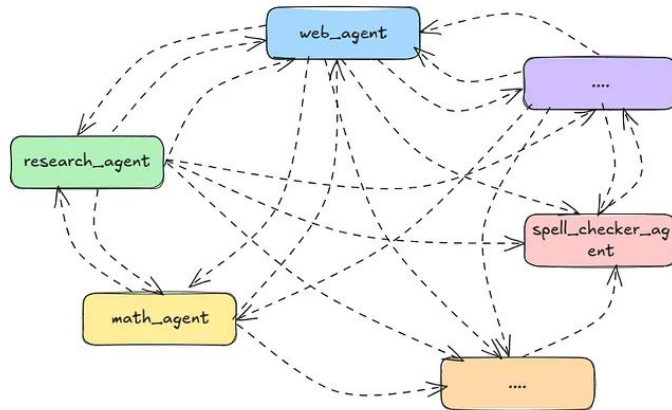
Supervision



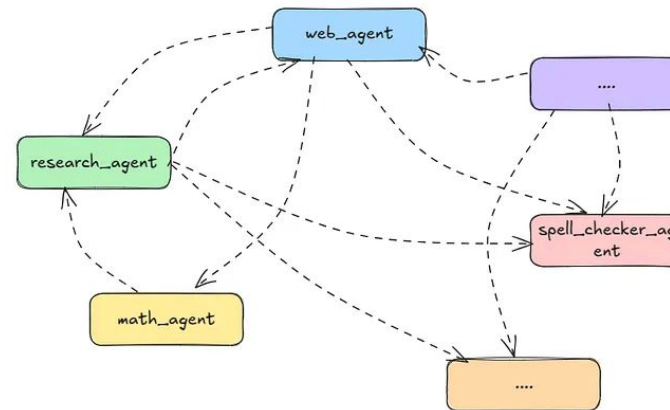
Hiérarchique



Réseau

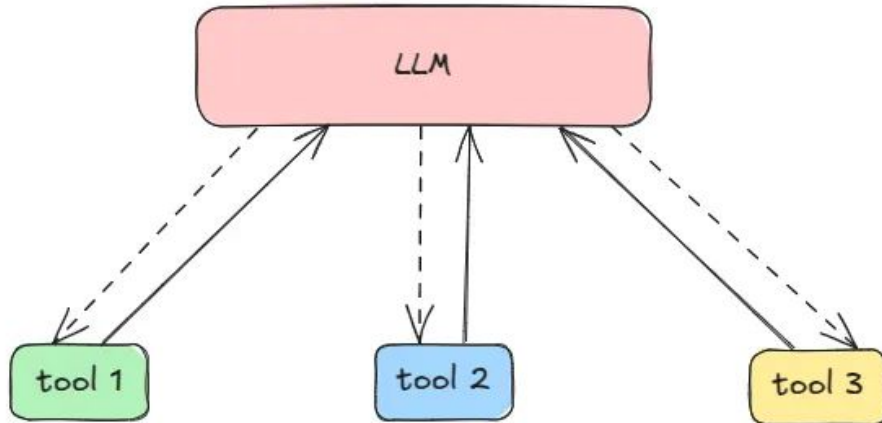


Personnalisé



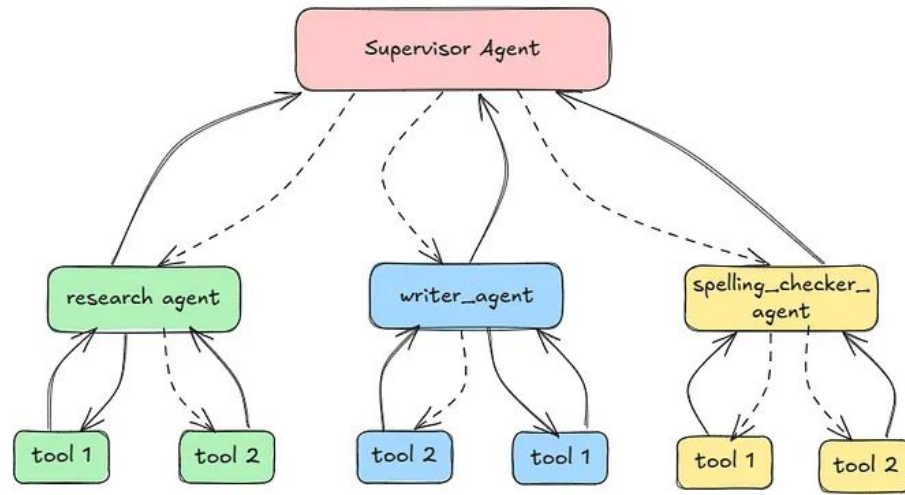
Source : [medium](#)

Architecture à agent unique (Single Agent)



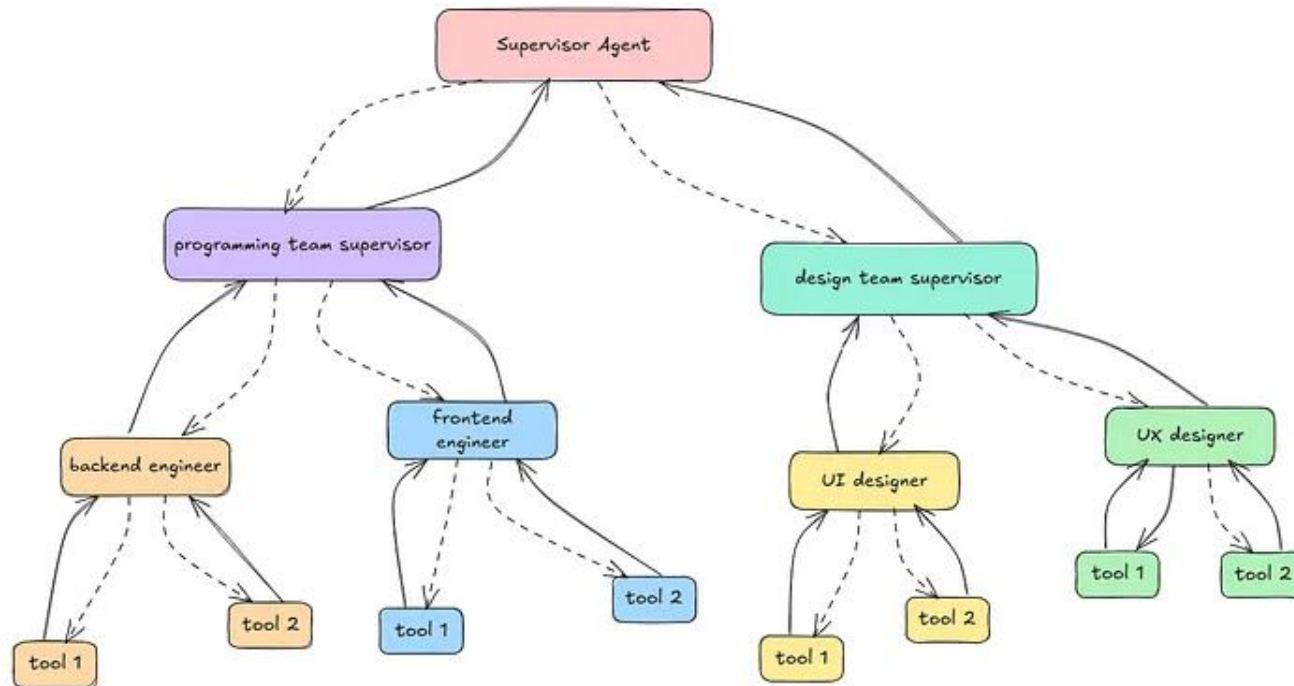
- Le LLM **planifie lui-même la séquence d'actions**
- Il **choisit les outils nécessaires** pour chaque étape
- Il **ajuste sa stratégie au fur et à mesure** en fonction des résultats intermédiaires
- Le tout avec une **intervention humaine minimale**

Architecture de supervision (Supervisor Agent)



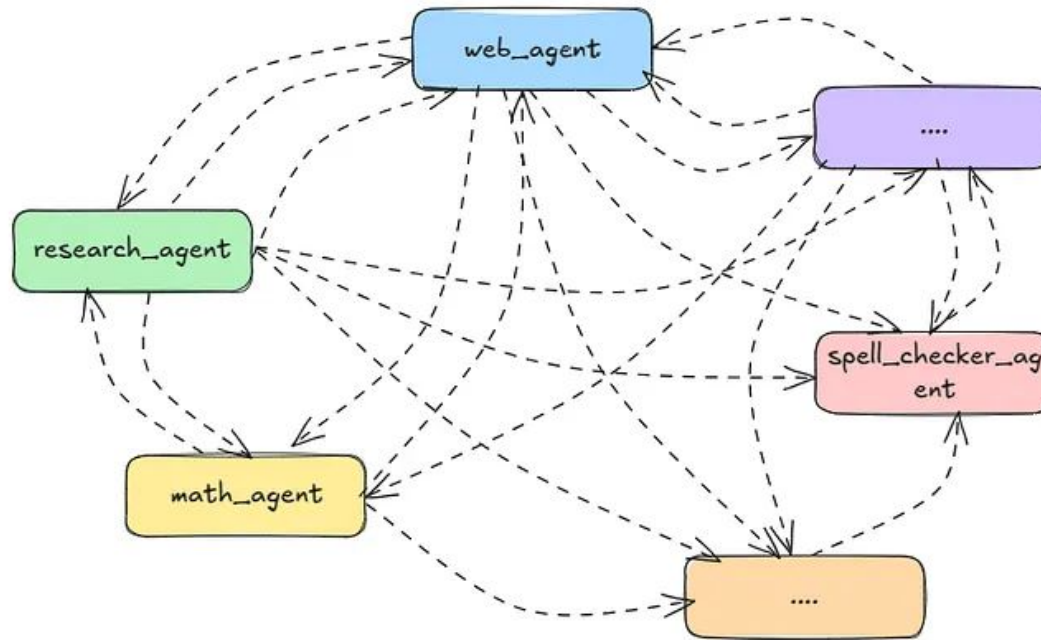
- Un **agent superviseur** coordonne plusieurs sous-agents
- Le superviseur agit comme un **chef d'orchestre**
- Il **reçoit les requêtes** et utilise un LLM pour raisonner
- Il **décide à quel agent déléguer** la tâche
- Il **récupère les résultats** produits par les sous-agents
- Il **évalue** s'il faut :
 - faire appel à un autre agent, ou
 - retourner la **réponse finale**

Architecture hiérarchique (Hierarchical Agent)



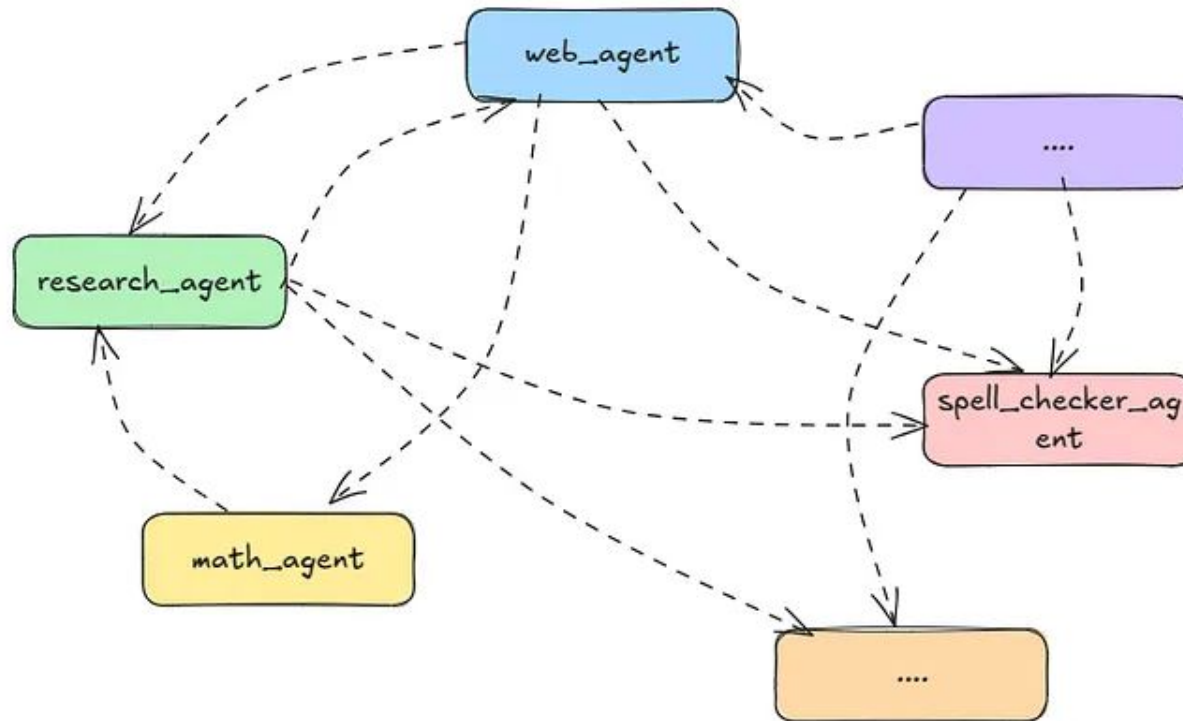
- Quand un **superviseur unique devient surchargé**
- L'approche évolue en **équipes d'agents**, chacune avec son **superviseur spécialisé**
- Le système se **divise en sous-graphes**
- Chaque sous-graphe possède :
 - un **superviseur dédié**
 - des **sous-agents spécialisés** dans des domaines particuliers

Architecture en réseau (Network Agent)



- Chaque agent peut **communiquer avec n'importe quel autre agent** du système
- Fonctionne comme un **réseau social** : possibilité de contacter librement n'importe qui
- Tous les agents sont **interconnectés**
- Ils peuvent **échanger librement** entre eux

Architecture personnalisée (Custom Agent)



- Contrairement à l'**architecture en réseau**
- Chaque agent ne peut **communiquer qu'avec des sous-agents sélectionnés spécifiquement**
- Cela crée un **réseau de communication plus restreint**
- L'échange est donc plus **contrôlé et limité**

Quand est-ce qu'il faut utiliser les multi-agents ?

Le piège de la sur-ingénierie IA

- **Phase "démonstration"** : Effet *WoW*, tout semble possible
- **Phase "réalité"** : Débogage cauchemardesque, coûts explosifs, comportements imprévisibles
- **Phase "maturité"** : Retour aux fondamentaux avec des choix architecturaux mesurés

Principe du parcours de complexité minimale

Monter l'**échelle agentique** étape par étape, seulement si nécessaire :

- ☆☆☆ **Script simple** → *Est-ce que ça marche ?*
- ★☆☆ **Workflow déterministe** → *Encore trop rigide ?*
- ★★☆☆ **Agent unique avec outils** → *Vraiment pas assez ?*
- ★★★ **Multi-agents** → *À envisager seulement après tout le reste*

Les vrais cas d'usage multi-agents

Selon mon expérience, ils se limitent vraiment à :

- Recherche exploratoire (quand on ne sait pas quelles pistes suivre)
- Émulation de processus humains complexes (négociation, diagnostic collaboratif)
- Systèmes adaptatifs dans des environnements très volatiles

Écosystème & front
(panorama outillage)

Typologie des solutions

	Frameworks multi-agents (logique d'équipe d'agents)	Solutions commerciales end-to-end (clé-en-main)	Orchestrateurs low-code / automatisation métier
Outils	CrewAI, AutoGen, LangGraph	Dialogflow CX, Microsoft Bot Framework, Cognigy	n8n, Make, Zapier
À quoi ça sert ?	Coordination d'agents avec rôles, tâches, dialogues ou logique conditionnelle. Orchestration de workflows complexes.	Interfaces graphiques, gestion des intentions, intégration avec call centers et systèmes métiers. Solutions clé-en-main.	Construction de workflows : CRM, email, notifications, enrichissement de données. Connecteurs vers APIs externes.
Cas d'usage	MVP intelligents, assistants IA spécialisés, logique métier modulaire nécessitant plusieurs agents collaboratifs.	Grandes entreprises avec parcours client bien définis, support client automatisé, intégration systèmes existants.	Connecter les outils, déclencher des actions, structurer le business autour de l'agent IA sans développement complexe.

Outils populaires

Outil / Brique	Rôle principal	Exemple d'usage
LangChain	Framework généraliste pour orchestrer prompts, outils et RAG	Construire un chatbot avec mémoire et accès documentaire
PydanticAI	Contraindre et valider les sorties LLM avec des schémas Pydantic	Extraction fiable d'entités dans un contrat ou une facture
Vector Stores (Qdrant, Weaviate, FAISS)	Indexation et recherche sémantique	RAG pour retrouver les passages pertinents dans une base documentaire
Observabilité (Langfuse,)	Traçage, monitoring et évaluation des appels LLM	Suivre les coûts, debugger et améliorer la qualité des réponses

Les frameworks **multi-agents** à considérer

	CrewAI	AutoGen	LangGraph
Popularité github	Github 38k	Github 49.7k	Github 18.5k
Difficulté		 	  
Coût	Gratuit (+version payante CrewAI Enterprise)	Gratuit	Gratuit (+version payante)
Langage	Python, Javascript	Python, .Net	Python
Développé par	Open source (CrewAI Inc.)	Microsoft Research	LangChain Team
Backend	LangChain	Python	LangChain
Ciblé pour	MVP startup, PoC, produit en croissance	Recherche, tests de dialogue entre agents	Architectures complexes, prod avancée

Zoom sur les orchestrateurs

Historiquement, ce sont des **outils d'automatisation** no-code / low-code
→ Aujourd'hui, ils intègrent des composantes IA (LLM, agents, plugins)
→ Leur rôle : connecter les agents aux outils métier (CRM, emails, reporting...)



Zapier

- Très accessible, parfait pour **débutants** et prototypes rapides
- Environ **6000** intégrations disponibles
- Limité pour les logiques complexes ou personnalisées
- Coût élevé à grande échelle
- Hébergement aux US → attention RGPD



Make

- **Bon compromis** entre simplicité et puissance
- Interface visuelle riche, logique conditionnelle (routes, boucles)
- Intégrations IA disponibles
- Tarification très avantageuse
- Hébergement cloud en Europe



n8n

- Le plus technique, mais aussi **le plus flexible**
- Open-source, self-hostable
- **Intégration de code** JS/Python, scénarios IA avancés (LangChain...)
- Idéal pour les besoins spécifiques ou sensibles (RGPD, infra interne)
- Nécessite plus de mise en place au départ

Les frameworks front

Pour un POC rapide en python

- **Streamlit** : très simple à prendre en main, parfait pour prototyper un assistant ou tester une intégration.
- **Gradio** : Interface plug-and-play, idéale pour démos rapides et partage via Hugging Face Spaces.

✓ Avantage : installation en minutes, zéro front-end requis.

⚠ Limite : pas pensé pour passer en production avec des UX complexes.



Pour un chat agentique clé en main

- **Chainlit** → Spécialisé pour les assistants et les workflows agentiques (support streaming, intégration directe avec LangChain/LangGraph).

✓ Avantage : solution dédiée aux LLM agents, rapide à connecter.

⚠ Limite : moins flexible qu'un vrai framework front si tu veux un design 100% custom.



Pour une app pro / SaaS

- **Next.js + Tailwind + Vercel AI SDK**
 - Stack moderne (React), parfaite pour un produit robuste et scalable.
 - Vercel AI SDK gère nativement les chats en streaming et tool calling côté front.

✓ Avantage : industrialisable, UX moderne, écosystème énorme.

⚠ Limite : investissement dev front nécessaire.



Autre framework à considérer



Les frameworks pour effectuer des outputs structurés

Noms	Instructor	Trustcall	Outlines
Popularité	Github 11k	Github 898	Github 12.6k
Idée globale / Features	Wrapper haut-niveau autour d'APIs LLM, on passe un modèle Pydantic et il gère validation + retries.	S'appuie sur la génération itérative de JSON par patches successifs plutôt que de demander au modèle de construire directement un gros JSON final.	Contrainte la génération pendant le décodage en traduisant un modèle Pydantic ou schéma JSON en grammaire (CFG) que le LLM doit suivre token par token
Avantages	• Simplicité d'usage (client OpenAI-like) • Multi-fournisseurs (OpenAI, Claude, Mistral, etc.) • Très populaire et maintenu	• Robuste sur gros schémas imbriqués • Tolère erreurs partielles • Moins coûteux qu'un retry complet	• Sortie toujours valide à 100% • Fonctionne avec modèles open source (Mistral, LLaMA...) • Rapide et efficace
Inconvénients	• Dépend du respect du LLM • Pas 100% garanti (retry nécessaire)	• Plus complexe (multi-étapes) • Dépendance à LangChain/LangGraph • Communauté limitée	• Pas dispo en API hébergée (ex. OpenAI natif**) • Doc limitée pour cas avancés • Apprentissage grammaire

***Note : on peut également faire du CFG en natif sur openAI*

LiteLLM

LiteLLM facilite le travail avec les modèles d'IA en créant un **LLM gateway**



Gestion des coûts et budgets

- Cost Tracking : suivi des dépenses en temps réel
- Budgets : contrôle et limitation des coûts par projet/utilisateur

Contrôle d'accès et sécurité

- Model Access : gestion des permissions d'accès aux modèles
- Guardrails : protection et filtrage du contenu
- Rate Limiting : limitation du nombre de requêtes

APIs et intégration

- Batches API : traitement par lots pour l'efficacité
- Pass-Through Endpoints : proxy transparent vers les APIs natives

Monitoring et observabilité

- LLM Observability : surveillance des performances des modèles
- s3 Logging : archivage des logs et données

Gestion des prompts

- Prompt Management : versioning et optimisation des prompts

Mon expérience avec les outils de développement *agentique*



Cursor - Mon outil actuel

- IDE basé sur un fork de VS Code avec IA intégrée
- Abonnement mensuel incluant 500 requêtes premium
- Système hybride : après épuisement, passage aux requêtes lentes ou consommation de crédits
- Interface familière pour une adoption immédiate
- Idéal pour un workflow quotidien fluide et sans friction

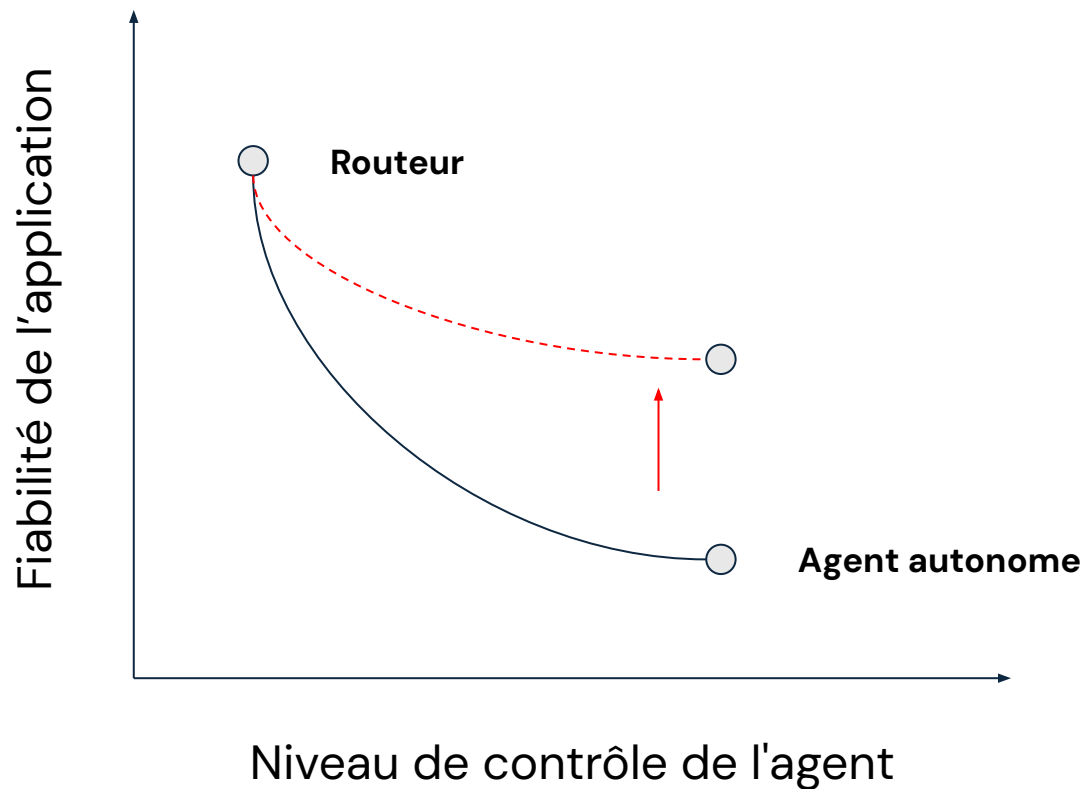


Claude Code - L'outil qui décolle

- Outil en ligne de commande d'Anthropic en pleine expansion
- Configuration initiale plus complexe mais potentiel énorme
- Très puissant une fois maîtrisé et correctement paramétré
- Flexibilité et contrôle avancés sur les interactions avec l'IA
- Parfait pour les développeurs recherchant une intégration profonde et personnalisée

Langgraph

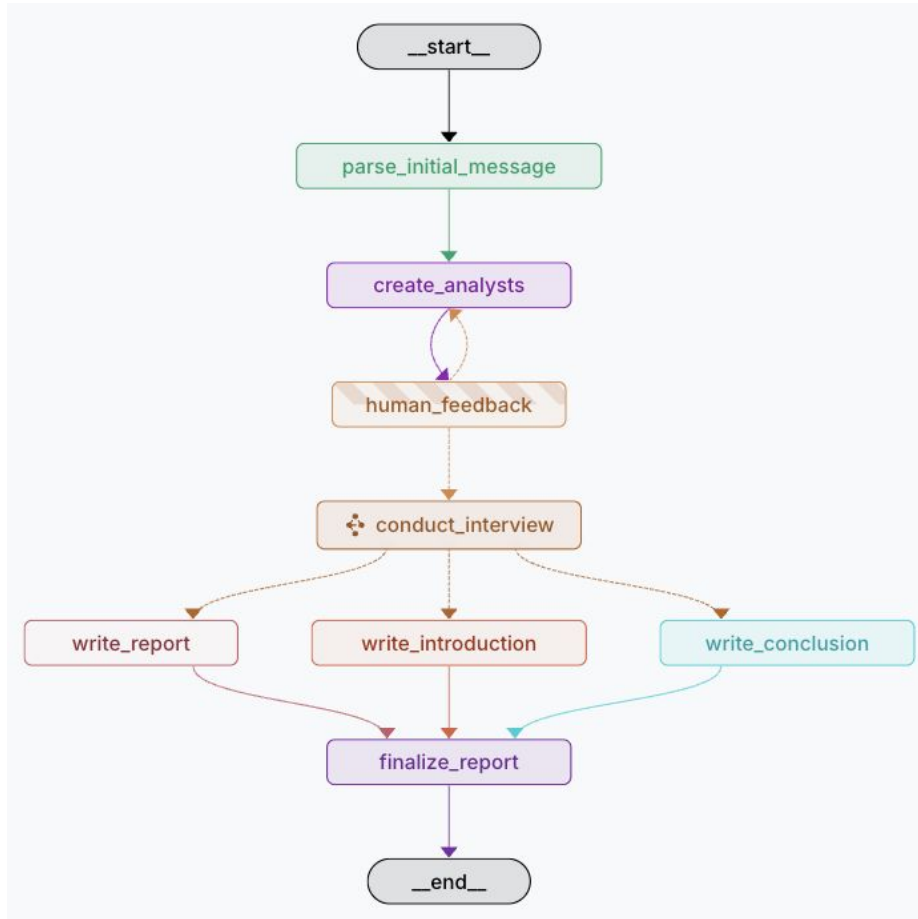
La promesse de LangGraph



Fiabilité des applications IA

- **Contexte :**
Quand on développe une application d'intelligence artificielle, l'enjeu principal est de la rendre fiable.
- **Problème :**
Plus on donne d'autonomie à un agent, moins l'application sera fiable.
- **Objectif :**
Gagner en fiabilité tout en profitant de la puissance des agents IA.

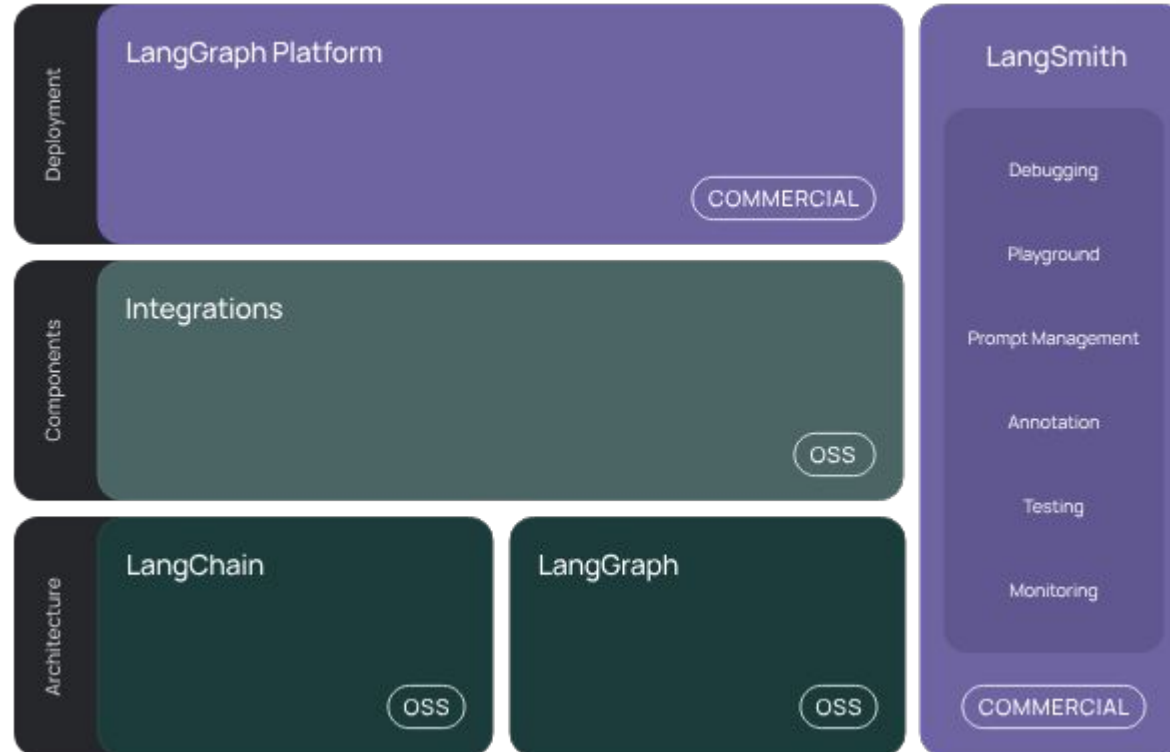
Fiabiliser le système avec LangGraph



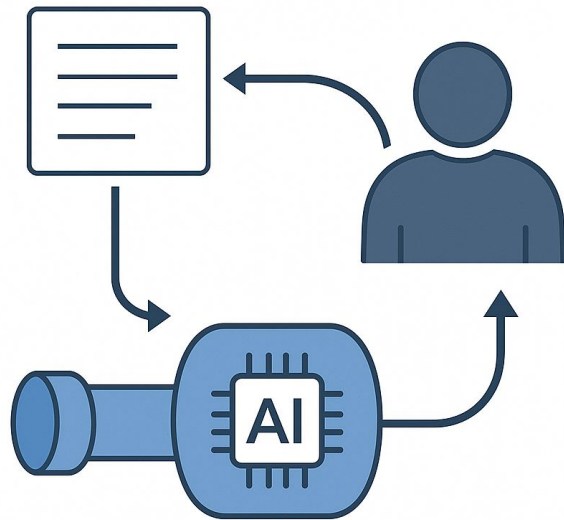
LangGraph est un framework qui permet de définir des **graphes** :

- **Un graphe** se compose de deux éléments :
 1. **Nœuds**
 2. **Connexions**
- **Les nœuds** contiennent des opérations basiques, de trois types :
 1. **Opération IA** : appel à un LLM, génération d'image, etc.
 2. **Opération fonctionnelle** : exécution de code (chargement de fichier, appel de fonction...)
 3. **Action humaine** : intervention ou validation manuelle
- **Avec ces 3 briques + des connexions**, on peut créer des architectures complexes et **mettre en place des gardes-fous**.

L'écosystème



Démonstration d'un workflow + illustration HIL



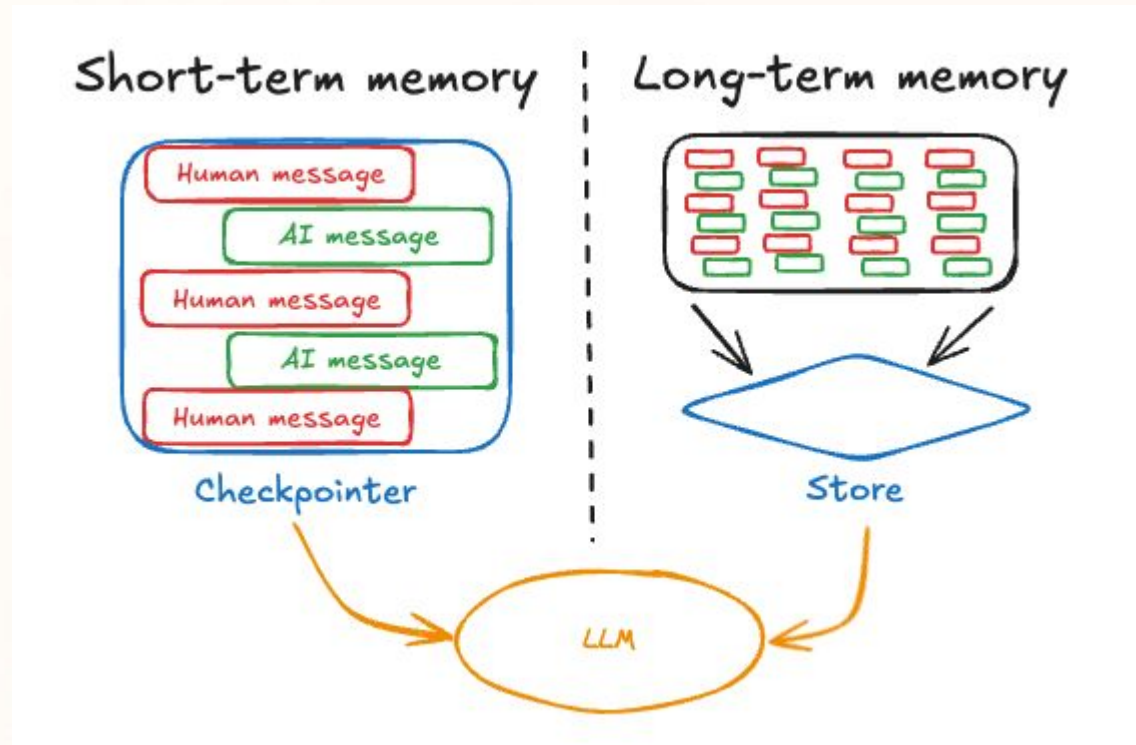
human-in-the-loop

Garantir fiabilité et contrôle grâce à une boucle continue de validation humaine

- **Principe**
L'IA propose, l'humain valide ou corrige avant exécution
- **Mécanismes clés**
 - **Points de contrôle**
 - Nœuds « validation humaine » à étapes critiques
 - Seuils de confiance déclenchant une revue manuelle
 - **Boucle de feedback**
 - Collecte des corrections pour réentraîner ou ajuster les prompts
 - Suivi des métriques qualité (taux d'acceptation, temps de validation)
 - **Escalade intelligente**
 - Détection automatique des cas ambigus ou sensibles
 - Alerte et distribution vers spécialistes

La mémoire

La mémoire



Mécanisme de mémoire intégré (2 niveaux) :

- **Mémoire courte** : utilisée dans l'échange en cours (contexte immédiat de génération)
- **Mémoire longue** : retours utilisateurs, paragraphes validés, styles rédactionnels préférés.

La mémoire à court terme

Le Problème

- **LLMs = Stateless** : Pas de mémoire persistante entre les appels
- **Contexte limité** : 200k tokens (ChatGPT), variations selon les modèles
- **Besoin** : Maintenir la cohérence conversationnelle

Solutions Populaires

1. Historique complet

- Renvoyer tout l'échange à chaque appel
- Simple mais limité par la taille du contexte

2. Filtrage sélectif

- Garder uniquement messages user/assistant
- Éliminer les appels d'outils et métadonnées
- Optimise l'utilisation du contexte

3. Résumés de conversation

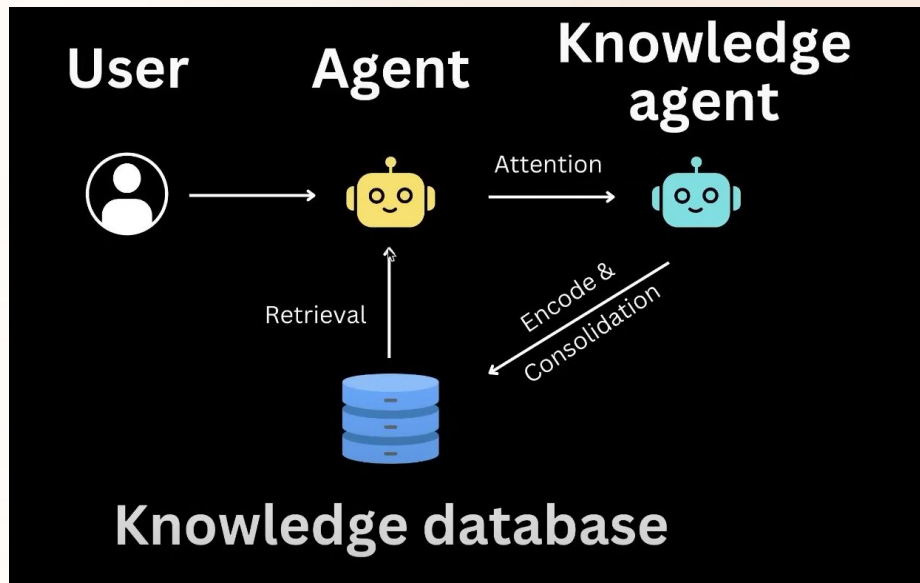
- Condenser les échanges anciens
- Préserver les informations essentielles
- Permet des conversations plus longues

La mémoire à long-terme : intuition

👤 “I’m Jason, I don’t eat fish”

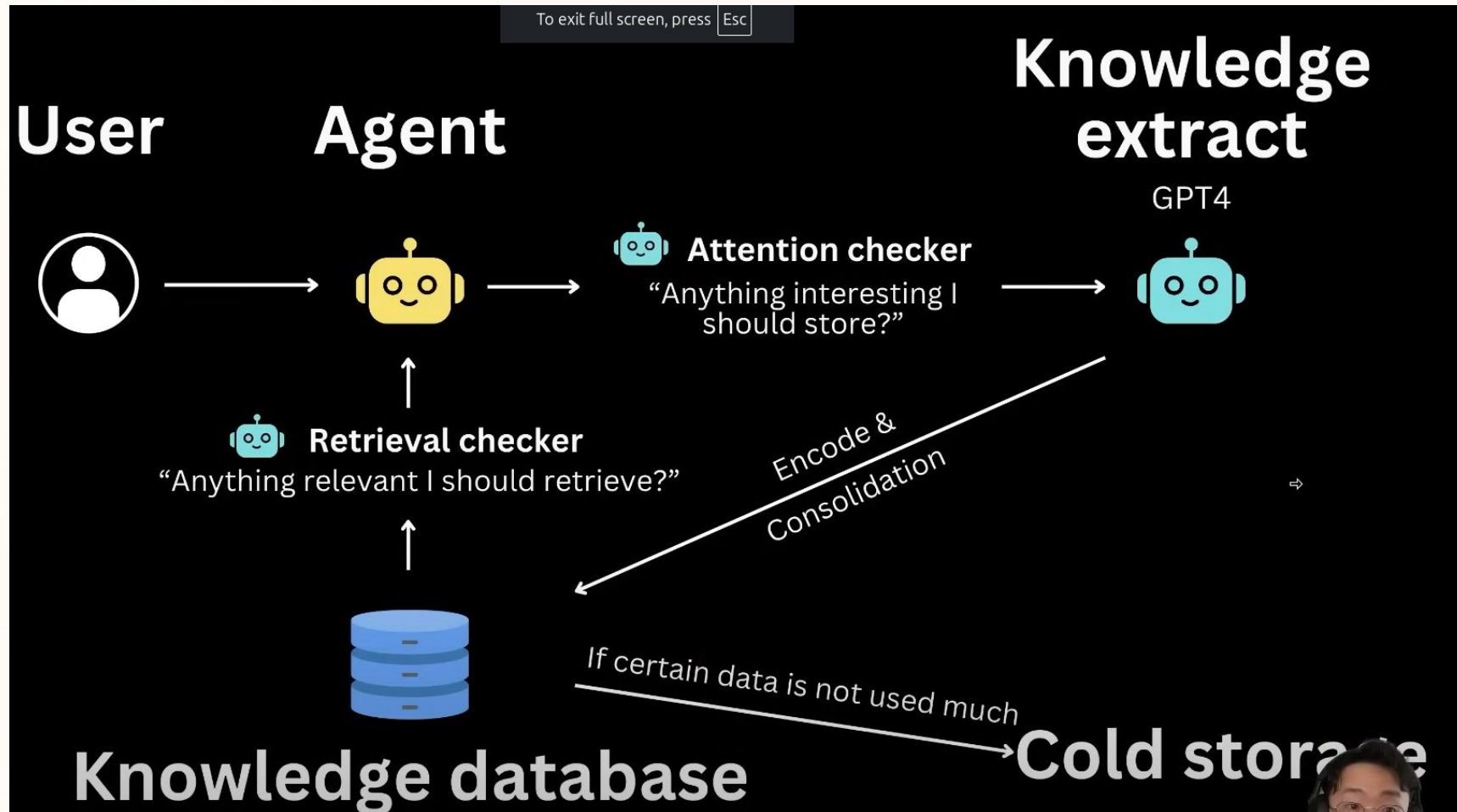
🤖 “Got it, here is a few meal options without fish...”

🤖 “Is there any info in the message worth saving as knowledge?”
-> If “Yes” ➡



source : AI Jason youtube

La mémoire à long-terme : intuition



source : AI Jason youtube

MemGPT: Towards LLMs as Operating Systems

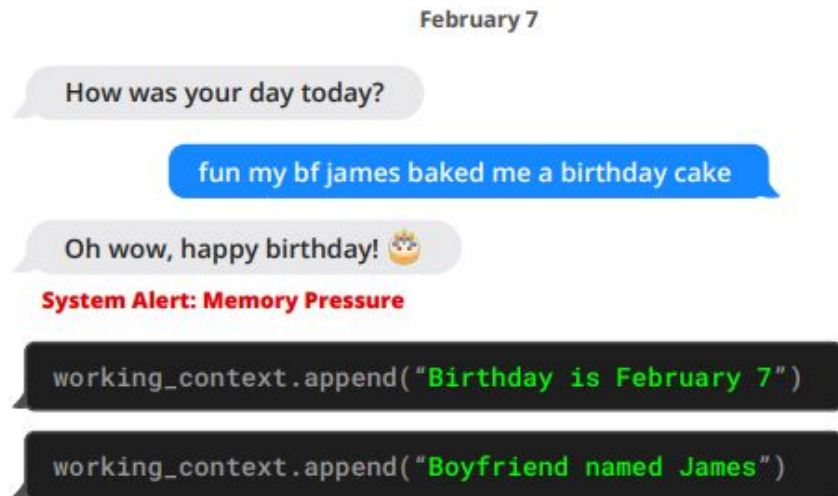


Figure 1. MemGPT (left) writes data to persistent memory after it receives a system alert about limited context space.

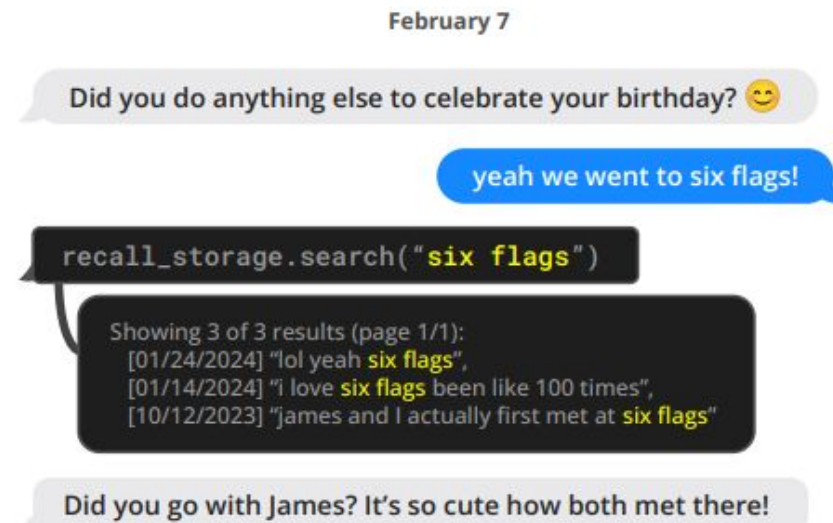


Figure 2. MemGPT (left) can search out-of-context data to bring relevant information into the current context window.

Sujets avancés & recommandations

MCP : Model Context Protocol

Le standard universel pour les APIs d'IA

Qu'est-ce que MCP ?

MCP (Model Context Protocol) est un nouveau standard pour construire des APIs dédiées aux modèles de langage (LLMs).

"L'USB-C des LLMs" - Une connexion universelle et standardisée

Architecture

Design et Origine

- **Créateur** : Anthropic
- **Architecture** : Client-Serveur
- **Transport** : Server-Sent Events (SSE) ou autres protocoles

Structure d'un serveur MCP

Un serveur MCP peut exposer deux types d'éléments :

1. **Ressources** - Données et contenus accessibles
2. **Outils** - Fonctionnalités et actions exécutables

Déploiement et Intégration

Côté Serveur

- Développement comme une API standard
- Déploiement sur n'importe quel hébergeur cloud
- Scalabilité et maintenance simplifiées

Côté Client

- **Intégration native** dans les éditeurs (ex: Cursor)
- **Compatibilité** avec de nombreux clients MCP
- **Plug & Play** - Connexion instantanée

Cas d'usage concret

Mon utilisation perso : DeepWiki + Cursor

Problème : Hallucinations lors de l'utilisation de documentation technique

Solution MCP :

1. Serveur MCP DeepWiki contient des **outils** pour lire la documentation
2. Ces outils accèdent aux **ressources** (la documentation réelle)
3. Cursor se connecte au serveur via MCP
4. L'IA utilise automatiquement les outils de lecture pour consulter la vraie documentation
5. Force le LLM à répondre en se basant sur les données réelles plutôt que sa mémoire

Résultat : Réponses précises et actualisées, réduction drastique des hallucinations

Outils (Actions exécutables)

- 🔍 **Recherche** - GitHub repos, Google Drive, bases de données
- 📝 **Création** - Issues GitHub, emails, documents
- 🤖 **Automatisation** - Puppeteer, tests, déploiements
- 📊 **Analyse** - Traitement CSV, génération de rapports
- 🌐 **Web scraping** - Collecte de données sites web

Ressources (Données contextuelles)

- 📁 **Systèmes de fichiers** - Accès aux dossiers et documents locaux
- 🗄️ **Bases de données** - Tables PostgreSQL, MongoDB, etc.
- 📖 **Documentation** - Wikis, manuels techniques, guides
- 💬 **Historiques** - Logs Slack, emails, conversations
- 🔗 **APIs externes** - Données météo, financières, etc.
- 📌 **Repositories de prompts** - Collections de templates réutilisables

Recommandation : Déploiement Entreprise

Le défi de la liberté MCP

- **Écosystème ouvert** – Un LLM peut se connecter à de nombreux serveurs MCP
- **Risques potentiels** – Sources non fiables, données sensibles, hallucinations

Solution : Serveur MCP d'Entreprise

Concept : Une "API Gateway" MCP centralisée

Architecture recommandée :

1. **Serveur MCP unique** – Point d'entrée contrôlé pour toute l'entreprise
2. **Ressources curées** – Intégration uniquement de sources fiables et validées
3. **Accès restreint** – Applications internes autorisées à interroger uniquement ce serveur
4. **Gouvernance** – Contrôle des données, sécurité, et qualité centralisés

Avantages :

-  **Sécurité** – Contrôle total des sources de données
-  **Fiabilité** – Élimination des sources douteuses
-  **Cohérence** – Standards uniformes dans l'organisation
-  **Audit** – Traçabilité complète des requêtes

Développer en interne vs externaliser

Ne pas surinvestir dans...

- Refaire des tools “génériques” bas niveau.
- Branches ad-hoc spécifiques à un vendor.
- Pipelines opaques sans export/contrat

Objectif. Être **data-centric** pour brancher facilement des outils externes (SaaS/OSS) sans réécrire la stack.

Concrètement, on investit dans :

- Contrats de données :
- Formats ouverts & portables
- Couches d'accès standardisées
- Gouvernance
- Documentation

Du prompt engineering au Context engineering

Le context engineering est le nouveau sujet à la mode en 2025 après les MCP

